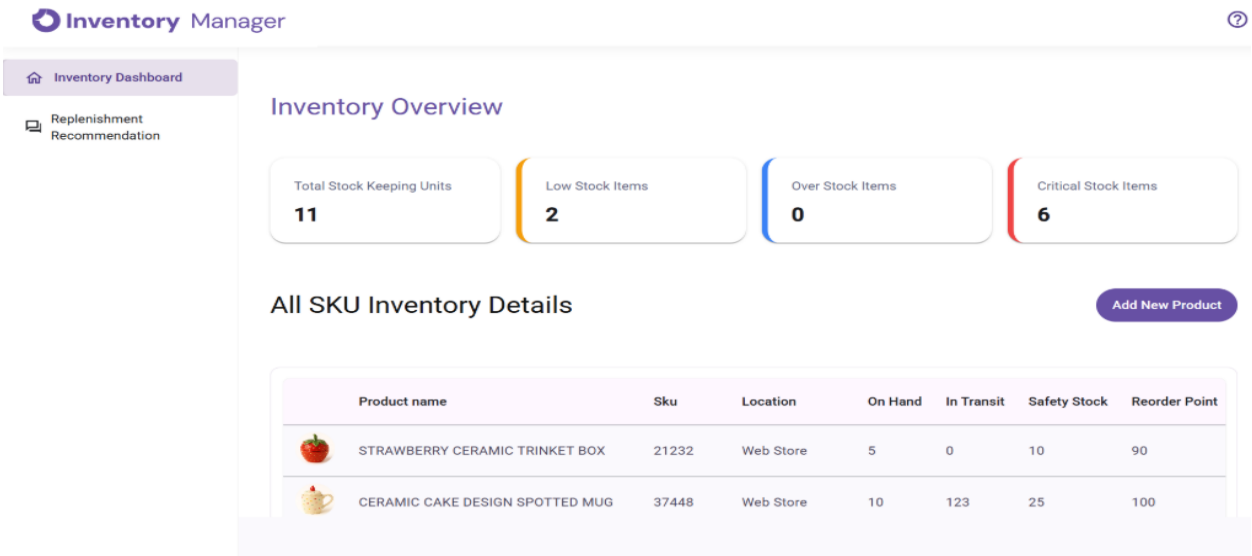
Inventory Management User Guide

Goal: Perform real-time, **policy-compliant** inventory replenishment whenever **stock_levels.on_hand** falls below the **reorder point** by coordinating specialized agents **Demand**, **Inventory**, **Supplier**, and **Policy** through a central **Coordinator**. The system leverages **AlloyDB**, **AlloyDBVectorStore**, and **Vertex AI embeddings** to retrieve supplier details and policy documents with citations, calculate **recommended order quantities** (accounting for **MOQ**, **caps**, and **substitutions**), generate a **purchase order (PO) draft**, and deliver an approval summary.

Workflow:

1. **Inventory trigger** initiates the process.
2. A **Coordinator** prepares the end-to-end plan.
3. **Demand** is evaluated from mv_daily_demand.
4. **Supplier** data is pulled (MOQ, lead time, cost) along with **RAG** over retail documents.
5. A **Policy Agent** performs **RAG with citations** to validate rules.
6. The system computes required **quantity**, applies **caps/MOQ**, and finalizes the **supplier** selection.
7. A **PO draft** is generated and users are **notified** to approve, edit, or reject.
8. (Optional) Deploy via **Cloud Run**, with **Pub/Sub** and **Scheduler** for automation.

Sample Images:



Inventory Manager

- Inventory Dashboard
- Replenishment Recommendation

Inventory Overview

Total Stock Keeping Units	Low Stock Items	Over Stock Items	Critical Stock Items
11	2	0	6

Low Stock Items

	Product	SKU	Location	On Hand	In Transit	Safety Stock
	MINI CAKE STAND HANGING STRAWBERY	22055	New York Store	13	80	10
	TEA TIME TEA TOWELS	47556B	New York Store	31	10	25

Inventory Manager

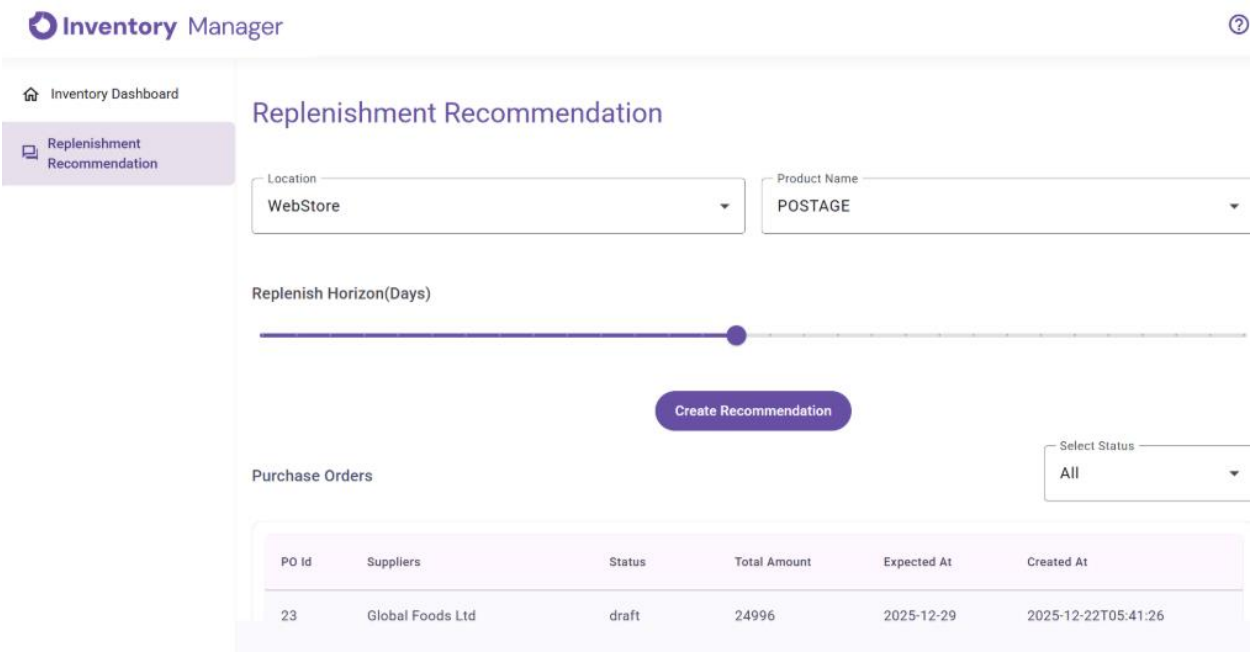
- Replenishment Recommendation

Inventory Overview

Total Stock Keeping Units	Low Stock Items	Over Stock Items	Critical Stock Items
11	2	0	6

Critical Stock Items

	Product	SKU	Location	On Hand	In Transit	Safety Stock
	STRAWBERRY CERAMIC TRINKET BOX	21232	Web Store	5	0	10
	CERAMIC CAKE DESIGN SPOTTED MUG	37448	Web Store	10	123	25
	Coffee	102345	Bangalore Store	24	12	30



Key MySQL Components:

- **MySQL** serves as the primary operational database, maintaining authoritative records for products, stores, stock levels, transactions, suppliers, and purchase orders.
- **AlloyDBVectorStore combined with VertexAIEmbeddings** enables RAG-based retrieval over documents such as supplier notes, seasonal memos, policies, and SOPs, returning results with citations.
  Note: VECTOR(d) columns must match the embedding model dimension — d = 768 for Vertex textembedding005.
- **ScaNN Index** is the recommended vector-search option at scale, configured with USING scann (...), tuned num_leaves, and optionally max_num_levels = 2, followed by an ANALYZE operation for optimization.
- **pgvector (Fallback Option):** Supports HNSW or IVFFlat indexing using the appropriate operator class (e.g., vector_cosine_ops) and the matching <=> similarity operator.
- **LangChain Integrations:**
  - AlloyDBEngine for secure, IAM-authenticated connection pooling
  - AlloyDBVectorStore + VertexAIEmbeddings for retrieval workflows
  - *(Optional)* SQLDatabaseToolkit for structured SQL interactions
- **Google Cloud Storage (GCS)** acts as the centralized repository for retail.docs, including policies, SOPs, seasonal notes, and ingestion artifacts.

- **Orchestration & Runtime Components** *(optional for production)*:
    - **Cloud Run** for hosting agent services and workflow endpoints
    - **Secret Manager** for securely storing non-IAM secrets

Technical Challenge & Business Impact:

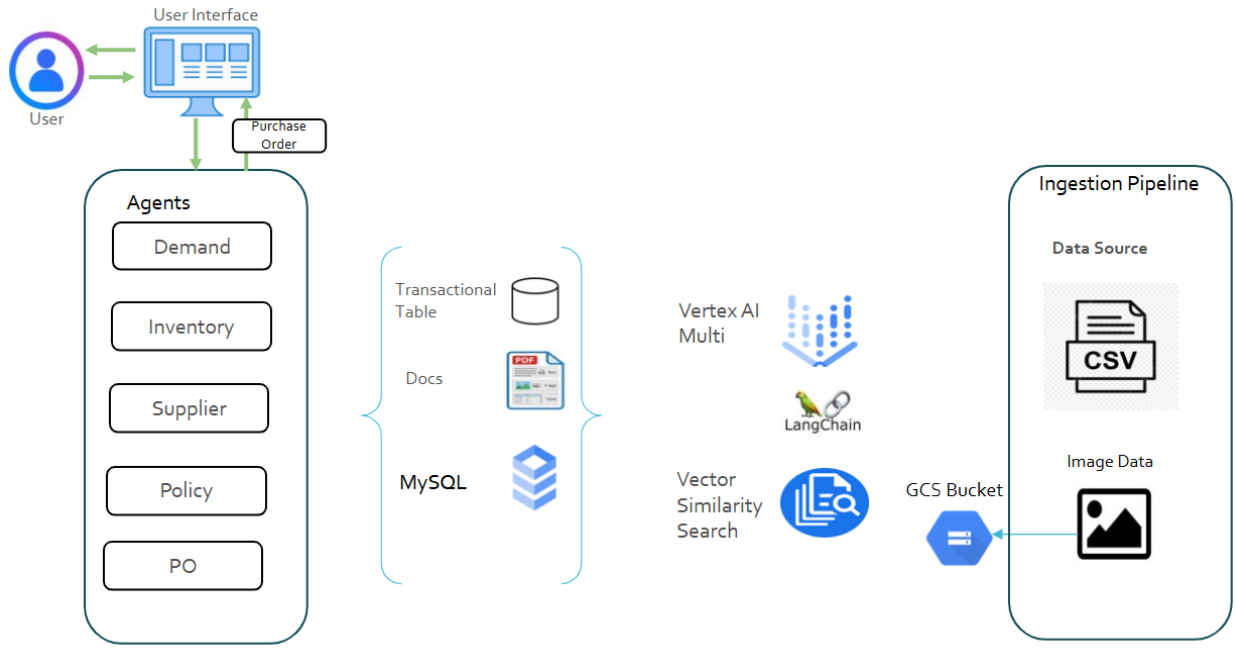
Challenge:

Manual, spreadsheet-based replenishment cannot keep up with rapid **fluctuations in demand,** supplier constraints (MOQ, lead time, cost), or **policy requirements** (caps, substitutions) at scale. Critical operational knowledge is often **scattered across documents**, resulting in slow, inconsistent decisions and limited **auditability**. Traditional systems also struggle to combine **real-time transactional data** with **policy-aware logic** and **explainable document retrieval**, leading to inefficiencies and missed opportunities.

Impact:

A real-time, **agent-driven**, **RAG-enabled** replenishment workflow built on **AlloyDB** and **GCP** automates **policy-compliant** purchase order generation with full **citation traceability**. This approach reduces **stockouts** and **excess inventory**, shortens the **decision-to-order cycle**, and improves overall **operational precision**. **Demand-based ordering, supplier-aware optimization** (MOQ, lead time, cost), and **policy-bounded recommendations** (caps, substitutions) together deliver higher **service levels**, improved **inventory turns**, and lower carrying costs while ensuring **audit-ready transparency** and **production-grade robustness**.

What the "Single Pass" Does: *(as implemented in src/workflows/run_single_pass.py)*

- Selects a frequent **(store, SKU)** pair from recent transactions and verifies that the required stock_levels and supplier mappings exist.
- Invokes the **Coordinator**, which executes the Demand, Inventory, Supplier, and Policy (RAG) agents using AlloyDBVectorStore and VertexAIEmbeddings.
- Calculates recommended_qty = max (0, (forecast × horizon + safety) – net_position), applies MOQ rounding, enforces policy caps, and determines the optimal supplier.
- Generates a purchase order draft, produces an approval summary (auto-approved in the demo workflow), writes the PO record, and outputs the generated **po_id**.

Architecture Diagram